SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

CO2 - ASSESSMENT TOOL - 3

ERROR ANALYSIS TASK

Course Code	**CSA3717**	**Course Title**	**Software Testing**
CO Assessed	**CO2**	**Assessment**	**Assessment Tool 3 : Error Analysis Task**
Weightage	**50% of CIA**	**Total Marks**	**100**
CO2 Statement	*Design effective test cases using black-box, white-box testing techniques and to control methods flow.***(BL3)**		
Student Name	**B Reddy Balu**	**Reg. No.**	**192312499**
Section/Batch	**Slot A / 2023**	**Date**	**04-11-2026**

Code of Conduct:

I ***B Reddy Balu (192312499),*** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

1. Introduction

Input validation is one of the first lines of defence in any software system. Every field that accepts data from a user, another system or a file must decide which values are acceptable and which must be rejected. When this decision is implemented incorrectly, the consequences range from minor user annoyance to corrupted databases, failed transactions and security vulnerabilities. Testing the validation logic therefore deserves a systematic approach rather than a handful of arbitrary inputs.

Because the number of possible inputs for even a simple field is practically infinite, exhaustive testing is impossible. Black-box test design techniques such as Equivalence Class Partitioning (ECP) and Boundary Value Analysis (BVA) address this problem. They help a tester select a small number of test inputs that still give a high probability of finding defects. These two techniques are described in the ISTQB Foundation Level syllabus and in standard textbooks on software testing, and they are the most widely used techniques for validating numeric input ranges.

This assignment studies a specific case. A tester applies ECP to an age field that should accept values from 0 to 100, but defines only two classes: negative numbers and positive numbers. The report (a) analyses the errors in this strategy, (b) explains why it fails to separate valid from invalid ages, (c) shows the consequences of the absence of Boundary Value Analysis, and (d) proposes a corrected set of partitions, with test cases and a justification of how they ensure more robust input validation.

2. Problem Statement and Assumed Specification

2.1 The question

A tester uses Equivalence Class Partitioning for an age-validation field (0–100) but defines only two classes: Negative numbers and Positive numbers. Analyse the errors in this partitioning strategy. Identify why this fails to account for valid vs. invalid age limits and the absence of Boundary Value Analysis. Propose a corrected set of partitions and justify how they ensure more robust input validation.

2.2 Assumed requirements

The question gives only the valid range. To design tests in a traceable way, the following requirements are assumed for the age field. These are reasonable interpretations and any different rule from a real specification can be handled by adjusting the affected classes.

Req. ID	Requirement	Rationale
R1	The field shall accept whole-number ages from 0 to 100, both inclusive.	Defines the valid class and both boundaries.

Req. ID	Requirement	Rationale
R2	The field shall reject any integer below 0 with a range error message.	Age cannot be negative.
R3	The field shall reject any integer above 100 with a range error message.	Upper limit of the business rule.
R4	The field shall reject non-numeric text (letters, symbols) with a format error.	Prevents type errors and injection.
R5	The field shall reject decimal values such as 25.5.	Age is captured as a whole number.
R6	The field shall reject empty or whitespace-only input with a "required" message.	Mandatory field behaviour.
R7	The field shall handle extremely large numbers without crashing.	Robustness against overflow.

Table 1: Assumed requirements for the age-validation field

3. Theoretical Background

3.1 Equivalence Class Partitioning

Equivalence Class Partitioning divides the set of all possible inputs into groups, called equivalence classes, such that the program is expected to behave in the same way for every member of a group. The reasoning is that if one member of a class reveals a defect, other members will probably reveal the same defect, and if one member passes, the rest are likely to pass as well. Therefore, one representative test value per class is theoretically sufficient to cover that class.

A correct partitioning must satisfy three properties:

- **Completeness (coverage):** every possible input belongs to at least one class, so no part of the input domain is left untested.
- **Disjointness:** no input belongs to more than one class, so classes do not overlap and test effort is not duplicated.
- **Homogeneity:** all members of a class are processed in the same way, so they are either all valid or all invalid for the same reason.

Classes are normally derived from the specification. For a range such as 0 to 100, the standard guideline is to create one valid class inside the range and two invalid classes outside it, one below and one above.

Additional invalid classes are created for other kinds of wrong input, such as the wrong data type or missing values.

Variants of ECP

In the literature, ECP is often described in four forms, depending on how classes are combined in test cases and whether invalid classes are tested:

Variant	**Description**
Weak normal	One representative from each valid class; only valid inputs are used.
Strong normal	Test cases for every combination of valid classes across multiple input variables.
Weak robust	Weak normal plus one test for each invalid class (one invalid value at a time).
Strong robust	Combinations of valid and invalid classes across all variables.

Table 2: Forms of Equivalence Class Partitioning

Because the age field has a single input variable, the weak robust form is the most suitable: it tests one valid class and every invalid class independently, which also makes defect diagnosis straightforward.

3.2 Boundary Value Analysis

Boundary Value Analysis is based on the observation that programmers make mistakes most often at the edges of ranges. Typical causes are the use of < instead of <=, off-by-one errors in loops and array indices, and confusion between inclusive and exclusive limits. BVA therefore selects test values exactly at the boundary and immediately on either side of it.

BVA approach	**Values selected for a range [min, max]**
Two-value BVA	min, max and the nearest invalid value just outside each limit: min−1, min, max, max+1.
Three-value BVA	min−1, min, min+1, max−1, max, max+1 (the boundary and both neighbours).
Robust BVA	Three-value BVA extended with values further beyond the range to test error handling.
Worst-case BVA	All combinations of boundary values across multiple variables (relevant only for several inputs).

Table 3: Common Boundary Value Analysis approaches

For the age field with min = 0 and max = 100, three-value BVA gives the set {−1, 0, 1, 99, 100, 101}.

4. Analysis of the Given Partitioning Strategy

The tester's strategy defines two classes: Class A, "Negative numbers" (values less than 0), and Class B, "Positive numbers" (values greater than 0). The following subsections evaluate this against the three ECP properties and against the specification.

4.1 The classes are not derived from the specification

The requirement states that valid ages lie in the range 0 to 100. The tester has instead partitioned by the mathematical sign of the number. Sign is not a property that the system's validation rule cares about. As a result, neither class is labelled valid or invalid, and no class corresponds to the central requirement (R1). A test design technique is only useful if its classes mirror the behaviour of the system under test; here, the classes describe a property that the software never checks.

4.2 The "Positive" class is heterogeneous

Class B contains values from 1 to 100, which the system must accept, and values from 101 upwards, which the system must reject. These two groups are processed differently, so they are not equivalent. This violates the homogeneity property, which is the very foundation of ECP. The practical consequence is that choosing any single representative from Class B gives a misleading result. If the tester picks 25, the test passes and suggests that the whole class works, yet the upper-limit check (R3) has not been exercised at all. If the tester picks 500, the test shows a rejection and suggests that positive numbers are rejected, which is equally misleading.

4.3 Zero is left out (incomplete partitioning)

Zero is neither negative nor positive. It belongs to neither Class A nor Class B, so the partitioning is not complete. This is more serious than an academic gap, because 0 is the lower boundary of the valid range and a very common site for defects. For example, an implementation using age > 0 would wrongly reject newborns (age 0), and the tester's suite would never detect it.

4.4 The mapping of sample inputs exposes the flaws

The table below classifies a range of inputs using the tester's partitions and compares the result with the correct system behaviour.

Input	Tester's class	Correct behaviour	Problem
−5	Negative	Reject (invalid)	Correctly classified
0	None (unclassified)	Accept (valid)	Falls outside every class
1	Positive	Accept (valid)	Correct, but class is mixed

Input	Tester's class	Correct behaviour	Problem
50	Positive	Accept (valid)	Correct, but class is mixed
100	Positive	Accept (valid)	Boundary not identified
101	Positive	Reject (invalid)	Same class as valid ages
150	Positive	Reject (invalid)	Same class as valid ages
abc	None	Reject (format error)	Input type ignored
25.5	Positive	Reject (not whole number)	Treated as valid by class
(empty)	None	Reject (required)	Missing input ignored

Table 4: Classification of sample inputs under the tester's two-class strategy

Out of ten inputs, only one is classified in a way that matches the system's behaviour without qualification. In three cases the input belongs to no class at all, and in four cases valid and invalid values share a class.

4.5 Failure to distinguish valid from invalid limits

The core purpose of validation testing is to check that the system accepts what it should accept and rejects what it should reject. The tester's partitions do not model this distinction. There is no valid class, and the invalid values above 100 are hidden inside a class that mostly contains valid values. On the lower side, the negative class is a legitimate invalid class, but without a matching class for values above 100 the validation is only half-specified. A well-designed partitioning for a bounded range should show one valid class and two invalid ones outside it, forming a clear "invalid | valid | invalid" structure; the tester's strategy shows "invalid | (gap) | mixed".

Region of number line	Correct partitioning	Tester's partitioning
Less than 0	Invalid class (below minimum)	Class A: Negative
Exactly 0	Part of the valid class	Not covered
1 to 100	Part of the valid class	Class B: Positive (mixed)
Greater than 100	Invalid class (above maximum)	Class B: Positive (mixed)

Table 5: Comparison of correct and tester's partitioning along the number line

4.6 Other invalid input types are ignored

A real input field does not receive only integers. Users may type letters, symbols, spaces, decimals, leading zeros, or nothing at all, and an attacker may enter a very long string or a number beyond the range of the underlying data type.

5. Consequences of the Flawed Strategy

A weak test strategy is not just a theoretical problem. It produces a false sense of confidence, because the test report shows passed cases while real defects remain. The following impacts are likely in a real project.

- **Undetected functional defects:** valid users aged 0 or 100 may be wrongly blocked, or invalid ages such as 101 may be stored, leading to wrong eligibility decisions in applications that depend on age (insurance, education, healthcare, voting, retirement benefits).
- **Data integrity problems:** incorrect values stored in the database corrupt reports and analytics and are expensive to clean later.
- **Security risk:** unvalidated text or oversized numbers can trigger exceptions, expose stack traces or, in poorly written systems, become a vector for injection attacks.
- **Poor user experience:** without separate range, format and required-field messages, users receive confusing or generic errors.
- **Higher cost of defects:** defects found in production cost far more to fix than those found in design or testing, so missing a simple boundary defect is an expensive saving.
- **Misleading metrics:** a test suite with few meaningful cases but a 100% pass rate can hide a low real defect-detection capability.

6. Proposed Corrected Partitioning

6.1 Design approach

The corrected partitioning follows four steps. First, identify the valid range from the specification (0 to 100). Second, create one valid class for that range. Third, create invalid classes for each distinct way the input can be wrong: below the minimum, above the maximum, wrong type, wrong format, missing, and beyond data-type capacity. Fourth, check that the classes are complete, disjoint and homogeneous. On the number line the integer domain is split as follows:

```
(-infinity ... -1]  |  [ 0 ... 100 ]  |  [101 ... +infinity)
  INVALID (EC1)     |  VALID (EC2)    |    INVALID (EC3)
```

Inputs that are not integers at all (text, decimals, blanks, overflow values) are handled by further invalid classes EC4 to EC7.

6.2 The corrected equivalence classes

ID	Class	Range / description	Type	Sample input	Expected result
EC1	Below minimum	Integers < 0	Invalid	−5	Rejected with range error
EC2	Valid age	Integers from 0 to 100	Valid	35	Accepted
EC3	Above maximum	Integers > 100	Invalid	150	Rejected with range error
EC4	Non-numeric	Letters, symbols, mixed text	Invalid	abc, 2a, @#	Rejected with format error
EC5	Non-integer	Decimal values	Invalid	25.5	Rejected with format error
EC6	Empty input	Null, blank, whitespace only	Invalid	(empty), " "	Rejected with "required" error
EC7	Extreme magnitude	Very large numbers beyond data-type limits	Invalid	99999999999	Rejected gracefully, no crash

Table 8: Corrected set of equivalence classes

6.3 Justification of each class

- **EC1 (below minimum):** retained from the tester's design because negative ages are a genuine invalid class, but now tied explicitly to requirement R2.
- **EC2 (valid age):** the central class that was missing. It allows the tester to confirm the acceptance behaviour required by R1, including the value 0 that was previously unclassified.
- **EC3 (above maximum):** separates the invalid upper values from the valid ones, which fixes the heterogeneity of the old Positive class and covers R3.
- **EC4 (non-numeric):** checks that type validation exists before range validation. A system that tries to compare text to a number may crash or behave unpredictably.
- **EC5 (decimals):** verifies that fractional ages are not silently truncated or rounded, which would store incorrect data.
- **EC6 (empty input):** verifies the mandatory-field behaviour. Empty input is a frequent source of null-pointer errors and may be incorrectly treated as zero.
- **EC7 (extreme magnitude):** verifies behaviour when the number exceeds the capacity of the data type (for example, above 2,147,483,647 for a 32-bit integer).

6.4 Sub-partitioning of the valid class (optional)

If the business rules apply different logic to different age groups, for example minors (0–17) and adults (18–100), the valid class should be split further so that each sub-class is tested, and the boundaries 17 and 18 are added to the BVA set. For the plain range validation described in this assignment, no such split is needed, because all ages from 0 to 100 are treated identically. Splitting a class without a difference in behaviour would only add unnecessary tests.

7. Boundary Value Analysis Test Cases

7.1 Boundary values for the valid range

Three-value BVA is applied to both ends of the valid class EC2. The lower boundary is 0 and the upper boundary is 100.

Test value	Position	Valid / invalid	Expected result
−1	Just below lower boundary	Invalid (EC1)	Rejected with range error
0	Lower boundary	Valid (EC2)	Accepted
1	Just above lower boundary	Valid (EC2)	Accepted
99	Just below upper boundary	Valid (EC2)	Accepted
100	Upper boundary	Valid (EC2)	Accepted
101	Just above upper boundary	Invalid (EC3)	Rejected with range error

Table 9: Three-value boundary test cases for the age field

Each of these six values targets a specific kind of mistake. The value −1 detects a lower limit set too low (for example >= −1). The value 0 detects a lower limit set too high (for example > 0). The values 1 and 99 confirm that the interior side of each boundary behaves correctly. The value 100 detects an upper limit that is exclusive when it should be inclusive (< 100), and the value 101 detects an upper limit set too high (<= 101) or missing altogether.

7.2 Data-type boundaries

Boundary analysis can also be applied to the technical limits of the input data type, particularly for the extreme-magnitude class EC7. For a signed 32-bit integer, the following values are useful for robust testing.

Value	Significance	Expected result
2147483647	Largest 32-bit signed integer	Rejected as out of range (above 100)
2147483648	One above the 32-bit maximum (overflow)	Rejected gracefully, no crash or wrap-around
−2147483648	Smallest 32-bit signed integer	Rejected as out of range (below 0)
−2147483649	One below the 32-bit minimum (underflow)	Rejected gracefully, no crash or wrap-around

Table 10: Data-type boundary values (robust BVA)

The wrap-around case is important. In some languages, adding one to the largest integer produces the smallest negative number, and a poorly written validator that parses to a fixed-width integer might misinterpret a huge input as a negative or small value and accept or reject it for the wrong reason.

7.3 Special format boundaries

A few additional edge cases concern the way the number is written rather than its numeric value. The expected behaviour must be agreed with the requirement owner, and the test then documents the decision.

Input	Issue	Suggested expected result
" 25 "	Leading and trailing whitespace	Accepted after trimming (or rejected, if the rule says so)
"025"	Leading zero	Accepted as 25, or rejected if strict format is required
"+30"	Explicit plus sign	Policy decision; document the result
"1e2"	Scientific notation (equals 100)	Rejected as invalid format
"٢٥" (Arabic-Indic digits)	Non-ASCII digits	Policy decision; commonly rejected
"-0"	Negative zero	Policy decision; usually treated as 0 and accepted

Table 11: Format-related edge cases to be clarified with the specification owner

8. Complete Test Suite

The final test suite combines one representative from each equivalence class with the boundary values. It uses the weak robust approach: one invalid condition is tested at a time so that a failure can be attributed to a single cause.

Test ID	Input	Technique / class	Requirement	Expected result
TC01	35	ECP: EC2 (valid)	R1	Accepted
TC02	−5	ECP: EC1	R2	Rejected: range error
TC03	150	ECP: EC3	R3	Rejected: range error
TC04	abc	ECP: EC4	R4	Rejected: format error
TC05	25.5	ECP: EC5	R5	Rejected: format error
TC06	(empty)	ECP: EC6	R6	Rejected: required field
TC07	99999999999	ECP: EC7	R7	Rejected, no crash
TC08	−1	BVA: below lower	R1, R2	Rejected: range error
TC09	0	BVA: lower boundary	R1	Accepted
TC10	1	BVA: above lower	R1	Accepted
TC11	99	BVA: below upper	R1	Accepted
TC12	100	BVA: upper boundary	R1	Accepted
TC13	101	BVA: above upper	R1, R3	Rejected: range error
TC14	" " (spaces)	ECP: EC6	R6	Rejected: required field
TC15	2147483648	BVA: data-type limit	R7	Rejected, no crash

Table 12: Complete test suite for the age-validation field

The suite contains fifteen test cases. This is a very small number compared with the infinite input space, yet every equivalence class, every boundary of the valid range and every requirement is covered.

9. Illustrative Implementation and Automated Tests

9.1 A validation function that meets the requirements

The following Python function implements requirements R1 to R7. It validates the format first and the range second, which is the same order in which the equivalence classes are checked.

```
def validate_age(raw):
    """Return (True, message) for valid age, else (False, reason)."""
    if raw is None or str(raw).strip() == "":
        return False, "Age is required"                # EC6

    text = str(raw).strip()
    if not text.lstrip("-").isdigit():
        # letters, symbols, decimals such as 25.5, 1e2 ...
        return False, "Age must be a whole number"      # EC4, EC5

    age = int(text)          # Python ints do not overflow
    if age < 0:
        return False, "Age cannot be below 0"           # EC1
    if age > 100:
        return False, "Age cannot be above 100"         # EC3, EC7
    return True, "OK"                                   # EC2
```

9.2 Automated tests derived from the suite

The test cases from Table 12 translate directly into a parameterised test. Each row is one test case, so adding a new case is a one-line change.

```
import pytest
from age import validate_age


@pytest.mark.parametrize("value, expected", [
    ("35", True),  ("-5", False), ("150", False),     # ECP
    ("abc", False), ("25.5", False), ("", False),
    ("99999999999", False), ("   ", False),
    ("-1", False), ("0", True), ("1", True),          # BVA
    ("99", True), ("100", True), ("101", False),
    ("2147483648", False),
])
def test_age_validation(value, expected):
    ok, _ = validate_age(value)
```

10. Evaluating the Effectiveness of the Two Strategies

10.1 Mutation-style comparison

A simple way to compare test suites is to deliberately introduce small defects (called mutants) into a correct implementation and check which suite detects them. The original strategy is represented by two typical tests, −5 and 50. The corrected strategy is the suite of Table 12.

ID	Injected defect	Observable effect	Original suite	Corrected suite
M1	Lower check changed to age > 0	Rejects 0	Missed	Detected (TC09)
M2	Upper check changed to age < 100	Rejects 100	Missed	Detected (TC12)
M3	Upper check changed to age <= 101	Accepts 101	Missed	Detected (TC13)
M4	Lower check changed to age >= −1	Accepts −1	Missed	Detected (TC08)
M5	Upper check removed	Accepts 150	Missed	Detected (TC03, TC13)
M6	No type check before conversion	Crash on "abc"	Missed	Detected (TC04)
M7	Decimal truncated to integer	Accepts 25.5 as 25	Missed	Detected (TC05)
M8	Empty input treated as 0	Accepts empty value	Missed	Detected (TC06, TC14)
M9	Overflow not handled	Exception on huge value	Missed	Detected (TC07, TC15)

Table 14: Defects detected by the original and corrected strategies

The original suite detects none of the nine injected defects (a mutation score of 0 out of 9), whereas the corrected suite detects all nine (9 out of 9). Although this is an illustration rather than a formal experiment, it shows clearly how the choice of partitions and boundary values determines whether defects are found.

10.2 Coverage comparison

Measure	Original strategy	Corrected strategy
Number of equivalence classes	2	7
Valid class defined	No	Yes (EC2)
Invalid classes defined	1 (negative)	6

Measure	Original strategy	Corrected strategy
Zero covered	No	Yes
Boundary values tested	0 of 6	6 of 6
Non-numeric / empty / decimal input tested	No	Yes
Requirements covered (of R1–R7)	1 partially	7 fully
Typical number of test cases	2	15

Table 15: Coverage comparison of the two strategies

The corrected strategy needs thirteen more test cases than the original, but the additional effort is modest and it is repaid many times over by the defects it finds early.

11. How the Corrected Partitions Ensure More Robust Input Validation

Robustness of validation means that the system behaves correctly for expected inputs, rejects unexpected ones safely, and never fails in an uncontrolled way. The corrected strategy supports this in the following ways.

11.1 Direct traceability to the requirements

Every class and every boundary test maps to a stated requirement (Table 13). When a test fails, the team knows exactly which requirement is violated, and when the requirements change, the affected tests can be identified quickly.

11.2 Clear separation of valid and invalid inputs

Because each class is homogeneous, a single representative really does stand for all members of its class. The tester can reason confidently that if 35 is accepted, all values in EC2 will be accepted, and if 150 is rejected with a range error, all values in EC3 will be too. This is the logical basis on which ECP reduces the number of tests without losing meaning.

11.3 Complete coverage of the input domain

The seven classes together cover every possible input: any value is either a valid integer in range, an integer below or above the range, non-numeric text, a decimal, empty, or beyond type limits. There are no gaps such as the unclassified zero in the original design, and no overlaps.

12. Limitations and Complementary Techniques

ECP and BVA are powerful, but not sufficient by themselves, and an honest evaluation should acknowledge their limitations.

- **Dependence on a correct specification:** if the requirement itself is wrong or ambiguous (for example, the maximum age is really 120), the tests will faithfully verify the wrong behaviour. Requirements should be reviewed together with test design.
- **Assumption of uniform behaviour inside a class:** if the implementation has hidden special cases (for example, a special rule for age 13), ECP can miss them. Error guessing and code review help here.

Therefore, the corrected partitions should be used as the foundation of the test design, supplemented by exploratory testing, error guessing based on past defects, and static review of the validation code.

13. Guidelines for Designing Partitions Correctly

The case study leads to a short checklist that testers can apply to any range-validated field.

1. **Start from the specification,** not from convenient mathematical categories.
2. **Identify at least one valid class** and then one invalid class for each distinct way the input can be wrong.
3. **Check the three properties:** completeness, disjointness and homogeneity of the classes.
4. **Look for the special values:** zero, empty, null, the minimum, the maximum and the type limits.
5. **Apply BVA to every ordered class edge,** with at least the values on and just outside the boundary.

14. Conclusion

The tester's decision to partition the age field into only Negative and Positive numbers is flawed in several fundamental ways. The classes are based on the sign of the number rather than on the validity rule of 0 to 100. The Positive class mixes valid ages with invalid ones above 100, violating homogeneity. Zero, the lower boundary, belongs to no class, violating completeness. Non-numeric, decimal, empty and oversized inputs are ignored, and no Boundary Value Analysis is performed. A suite built on such partitions can pass completely while the software contains defects in the most error-prone places.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand